RepoQA: A Retrieval-Augmented Question Answering System for Understanding GitHub Repositories

Sushmi Shrestha
Faculty of Advanced Science and Technology
Asian Institute of Technology
st126526@ait.asia

Abstract

Developers spend the majority of their time reading and understanding code, yet current AI coding assistants primarily focus on code generation and completion. When developers encounter an unfamiliar repository, they face questions about architecture ("How does authentication work here?"), code logic ("What does this middleware do?"), and deployment ("How is this service configured?") that existing tools cannot reliably answer. This **RepoQA**, a retrieval-augmented question answering system that helps developers understand GitHub repositories through natural language interaction. The approach introduces three key innovations: (1) a hierarchical summarization pipeline that generates project-, directory-, and file-level descriptions to enable multi-level understanding; (2) a hybrid retrieval engine combining semantic search, BM25 keyword matching, and dependency-graph traversal with question-type-aware routing; and (3) structured explanation prompts that produce grounded, citation-rich answers. The evaluation of RepoQA on the CoReQA benchmark (1,563 QA pairs from 176 repositories across four programming languages) using the LLM-as-a-judge framework. The experiments aim to demonstrate that hierarchical hybrid retrieval significantly outperforms the BM25 baseline reported in prior work (accuracy: 6.91/10, completeness: 6.34/10) and investigate the contribution of each architectural component through comprehensive ablation studies. This work addresses the critical gap between code generation tools and developer comprehension needs, advancing the state of repository-level code understanding.

1 Introduction

Software development is fundamentally a comprehensive activity. Studies consistently report that developers spend 58–70% of their time reading and understanding existing code rather than writing new code [Xia et al., 2018]. When onboarding to an unfamiliar repository, developers face a barrage of questions: *How is this project structured? What does this function do? Why was this design pattern chosen? How do I deploy this service locally?* These questions span multiple levels of abstraction—from high-level architecture down to individual function logic—and frequently require tracing dependencies across multiple files.

Despite rapid advances in AI-powered coding tools, the current landscape exhibits a fundamental mismatch. Tools such as GitHub Copilot [Chen et al., 2021], Cursor excel at code completion and generation but are not designed for code understanding and explanation. On the research side, most benchmarks target code generation [Chen et al., 2021, Austin et al., 2021] or bug fixing [Jimenez et al., 2024], with limited attention to question answering. Recent work by Chen et al. [2025] demonstrates this gap starkly: even GPT-4o with BM25 retrieval scores only 6.91/10 on accuracy for repository-level QA, and completeness scores languish at 6.34/10. The authors attribute this to BM25's inability to capture semantic relationships in code, where terms exhibit minimal variation and keyword matching misses crucial context.

To address this gap, the **RepoQA**, a retrieval-augmented question answering system designed specifically for repository-level code comprehension. RepoQA introduces three innovations. First, develop a *hierarchical summarization pipeline* that generates project-, directory-, and file-level summaries, enabling the system to answer questions at multiple levels of abstraction (following Tufek et al., 2025). Second, design a *hybrid retrieval en-*

gine that fuses semantic search, BM25, and dependency graph traversal, with a question-type router that selects the appropriate retrieval strategy based on whether the question targets architecture, logic, or deployment. Third, employ *structured explanation prompts* that organize context top-down (project summary → directory summary → code chunks) and require the LLM to cite specific files and line numbers, directly optimizing for the accuracy and completeness dimensions identified as challenging by Chen et al. [2025].

This paper makes the following contributions:

1. Formalize the task of repository-level question answering with a taxonomy of developer questions (architecture, logic, deployment) and propose a multi-level retrieval strategy tailored to each question type.
2. Design a complete RAG pipeline with hierarchical summarization, hybrid retrieval, and structured prompt generation for code explanation.
3. Provide a comprehensive evaluation plan using the CoReQA benchmark with LLM-as-a-judge evaluation across four quality dimensions, including ablation studies isolating the contribution of each component.

This work is guided by the following three research questions:

RQ1: Does hierarchical hybrid retrieval (semantic + BM25 + structural) significantly improve answer quality over BM25-only retrieval for repository-level code QA?

RQ2: How does question-type-aware routing (architecture vs. logic vs. deployment) affect retrieval precision and answer completeness compared to a uniform retrieval strategy?

RQ3: What is the individual contribution of each system component (hierarchical summaries, hybrid fusion, query expansion, structured prompts) to overall QA performance?

The hypotheses are – (1) hybrid retrieval will outperform BM25 by at least 0.5 points on accuracy and completeness (CoReQA scale), driven by semantic search capturing conceptual relevance that BM25 misses; (2) question-type routing will most benefit architecture questions, which require broad context, while logic questions will benefit most from function-level retrieval; and (3) hierarchical summaries will have the largest individual impact on completeness, as they provide the multi-level context needed to address all aspects of a question.

2 Related Work

2.1 Code Question Answering

Early code QA research focuses on snippet-level understanding. Liu and Wan [2021] introduce CodeQA, generating QA pairs from code comments at the function level, while CS1QA [Lee et al., 2022] targets introductory programming education. These benchmarks are limited to isolated code fragments and do not address the cross-file reasoning required in real-world repositories.

Recent work has shifted toward repository-level QA. Chen et al. [2025] introduce CoReQA, constructing 1,563 QA pairs from GitHub issues and comments across 176 repositories in four languages. Their evaluation reveals that even GPT-4o scores below 7/10 on accuracy, with BM25 providing negligible improvement. Peng et al. [2025] propose SWE-QA, specifically targeting module-level architectural comprehension—understanding how modules interact, their architectural roles, and semantic contracts between them. Strich et al. [2024] present a self-alignment pipeline for repository code QA at ACL 2024, using RAG over the Spyder IDE codebase. CodeRepoQA [CodeRepoQA Authors, 2024] offers a complementary multi-turn benchmark derived from repository discussions.

Gap: While these benchmarks define the problem, none propose a comprehensive retrieval solution. CoReQA identifies BM25's inadequacy but does not explore alternatives. SWE-QA introduces an agent-based approach but lacks ablation analysis. The work directly addresses the retrieval gap by proposing and evaluating a hierarchical hybrid pipeline.

2.2 Retrieval-Augmented Generation for Code

RAG has become the dominant paradigm for augmenting LLMs with external knowledge [Gao et al., 2024]. In the code domain, RepoCoder [Zhang et al., 2023] introduces an iterative retrieval-generation loop for code completion, while DraCo [Cheng et al., 2024] uses dataflow-guided retrieval. CAST [CAST Authors, 2025] demonstrates that AST-aware chunking outperforms fixed-size chunking for code RAG. CodeRAG-Bench [Wang et al., 2025] provides a systematic evaluation of 10 retrievers and 10 LMs on code generation tasks.

Gap: These approaches optimize retrieval for code *completion*, not code *understanding*. The retrieved context for completion (next token pre-

diction) differs fundamentally from what is needed for explanation (architectural context, design rationale, cross-file dependencies). The work adapts RAG for the explanation task through hierarchical summarization and question-type routing.

2.3 Repository-Level Code Understanding

Tufek et al. [2025] propose hierarchical summarization that constructs project-, directory-, and file-level summaries for bug localization, achieving state-of-the-art results by guiding LLMs through a top-down search. CodePlan [Bairi et al., 2024] frames repository-level tasks as planning problems but targets code editing, not comprehension. Gistify [Gistify Authors, 2025] evaluates codebase-level understanding through execution-based metrics but does not propose a retrieval method.

Gap: Hierarchical summarization has been applied to code search and bug localization, but not to developer-facing QA. Adapt and extend this approach specifically for the question-answering setting, combining it with hybrid retrieval and structured explanation prompts.

3 Methodology

Figure 1 presents the overall architecture of RepoQA, which consists of four main components: (1) repository ingestion and hierarchical summarization, (2) hybrid retrieval with question-type routing, (3) structured prompt generation, and (4) answer generation with grounding verification.

3.1 Repository Ingestion & Hierarchical Summarization

Given a GitHub repository R, we first clone it and traverse its file tree, filtering by language extensions and excluding generated artifacts (e.g., `node_modules`, `build/`).

Code Parsing. Use Tree-sitter to parse each source file into an Abstract Syntax Tree (AST) and extract semantic units: functions, classes, module-level code, docstrings, and comments. Following CAST Authors [2025], we employ AST-aware chunking rather than fixed-size sliding windows, ensuring each chunk respects syntactic boundaries.

Non-Code Indexing. Create index of non-code files critical for architecture and deployment questions: `README.md`, `Dockerfile`, CI/CD configurations, `package.json`, and environment files. These are chunked using standard text splitting.

Hierarchical Summarization. Inspired by Tufek et al. [2025] – generate summaries at three levels using an LLM:

- **Project-level:** A 1–2 paragraph summary of the repository's purpose, architecture, and key modules, generated from the README and top-level directory listing.
- **Directory-level:** For each top-level directory, a summary describing its role, key files, and relationship to other directories.
- **File-level:** For each source file, a summary of its exports (classes, functions), dependencies, and purpose.

Embedding & Storage. All chunks (code, non-code, and summaries at each level) are embedded using `all-mpnet-base-v2` and stored in ChromaDB with metadata (file path, chunk type, directory, language, summary level).

Dependency Graph. Construct an import graph $G = (V, E)$ where nodes V represent source files and edges E represent import relationships. This enables structural retrieval: if file A is relevant and imports file B, then B is likely also relevant.

3.2 Hybrid Retrieval with Question-Type Routing

Question Classification. Classify each developer question into one of three categories using a lightweight LLM prompt:

- **Architecture**: Questions about project structure, module relationships, design patterns (e.g., "How does authentication work?").
- **Logic**: Questions about specific function behavior, algorithms, or data transformations (e.g., "What does `validate_token()` do?").
- **Deployment**: Questions about configuration, build process, environment setup (e.g., "How is Docker configured?").

Retrieval Strategy Routing. Each question type triggers a different retrieval strategy:

- Architecture $\rightarrow$ Start with project and directory summaries, then expand to file-level chunks in identified modules (top-down).

- Logic → Start with function-level chunks via semantic search, then include caller/callee context from the dependency graph.
- Deployment → Prioritize non-code files (Dockerfiles, CI/CD, configs), supplemented by relevant code handlers.

Hybrid Fusion. Within each strategy, proposed to combine three retrieval signals:

1. **Semantic search:** Cosine similarity over embedded chunks (top-k=15).
2. **BM25:** Keyword matching over raw code text, essential for exact identifier names.
3. **Structural search:** Dependency graph traversal—if chunk from file A is retrieved and A imports B, boost B's chunks.

Results are fused using Reciprocal Rank Fusion (RRF):

$$\text{RRF}(d) = \sum_{r \in \{S,B,G\}} \frac{1}{k + \text{rank}_r(d)} \quad (1)$$

where S, B, G denote semantic, BM25, and graph-based rankings respectively, and $k = 60$ is a smoothing constant.

Query Expansion. Before retrieval, we use the LLM to expand the developer's question into additional search terms. For example, "How does auth work?" expands to include {`authentication`, `login`, `token`, `session`, `middleware`}.

3.3 Structured Prompt Generation

Retrieved context is organized into a structured prompt designed for explanation (not code completion):

1. **System instruction:** Role definition ("You are an expert on the {repo_name} codebase. Explain using ONLY the provided context. Cite file paths and line numbers.").
2. **Project summary:** High-level repo description.
3. **Directory summaries:** Summaries of relevant modules.
4. **Code chunks:** Retrieved code with file path headers
5. **Developer question:** The original question.

This top-down ordering mirrors how a human senior developer would explain code: starting with the big picture, then zooming into details.

3.4 Answer Generation & Grounding Verification

The LLM generates a structured answer with: (a) a brief overview, (b) detailed explanation with code snippets, and (c) file references. Post-generation, a *citation verifier* checks that every cited file path and function name exists in the repository. Invalid citations are flagged or removed, mitigating hallucination.

4 Datasets

4.1 Primary Evaluation: CoReQA

This paper adopts CoReQA [Chen et al., 2025] as the primary evaluation benchmark. CoReQA contains 1,563 QA pairs derived from GitHub issues and comments across 176 repositories in four programming languages (Python: 439, Java: 209, Go: 371, TypeScript: 544). Questions include mixed natural language and code snippets, and reference answers are constructed from community-validated issue discussions (requiring $\geq$3 positive reactions). Each QA pair includes 10 BM25-retrieved reference contexts.

4.2 Secondary Evaluation: SWE-QA

SWE-QA [Peng et al., 2025] provides architectural comprehension questions specifically targeting cross-module understanding. The experiment is it to evaluate RQ2 (question-type routing effectiveness), as its questions specifically require multi-file reasoning.

4.3 Development Repositories

For system development and debugging, select 1–3 open-source repositories of moderate size (5K–50K lines) spanning Python and TypeScript, including at least one repository the development team is unfamiliar with.

4.4 Data Preprocessing

For each repository, preprocessing involves: (1) cloning and file-tree filtering (excluding generated files, binary assets, test fixtures exceeding 1000 lines); (2) AST-based parsing and chunking with Tree-sitter, targeting chunks of 256–512 tokens; (3) hierarchical summary generation at three levels; (4) embedding all chunks and summaries; and (5) constructing the import dependency graph.

5 Evaluation Metrics

The experiment uses CoReQA's LLM-as-a-judge framework [Chen et al., 2025], which evaluates generated answers across four dimensions on a 1–10 scale:

- **Accuracy (Acc):** Factual correctness compared to the reference answer.
- **Completeness (Cmpt):** Whether all key aspects of the question are addressed.
- **Relevance (Rel):** Whether the answer addresses the core concern.
- **Clarity (Clar):** Logical structure and readability.

The paper additionally report **Pairwise Comparison Evaluation (PCE)** using Elo scoring for inter-system comparison, **Retrieval Recall@**k (k=5, 10) to measure retrieval quality independently of generation, and **Citation Accuracy** (fraction of cited file paths that exist in the repository).

Each LLM-as-a-judge evaluation is executed 3 times and we report mean $\pm$ standard deviation to account for judge variability ($\approx$5.5% per CoReQA's validation).

6 Experiment Design

6.1 Baselines

- **No Retrieval:** LLM answers using only the question (lower bound).
- **BM25 Only:** LangChain chunking + BM25 retrieval (CoReQA's reported baseline).
- **Semantic Only:** Embedding-based retrieval without BM25 or structural context.
- **Hybrid (no summaries):** Semantic + BM25 + structural, without hierarchical summaries.
- **RepoQA (full):** Complete system with all components.

6.2 Ablation Study (RQ3)

To isolate each component's contribution, we run the following ablations, each removing one component:

Configuration	Summ.	Hybrid	QExp.	Route
Full RepoQA	✓	✓	✓	✓
− Summaries		✓	✓	✓
− Hybrid (semantic only)	✓		✓	✓
− Query expansion	✓	✓		✓
− Type routing	✓	✓	✓	
BM25 baseline				

Table 1: Ablation configurations for RQ3.

6.3 Expected Results

Based on CoReQA's reported baselines and our preliminary experiments, we anticipate results in the ranges shown in Table 2.

System	Acc	Cmpt	Rel	Clar
No Retrieval	6.85	6.27	7.81	8.40
BM25 (reported)	6.91	6.34	7.86	8.42
RepoQA (expected)	**7.4–7.8**	**6.8–7.2**	**8.0–8.3**	**8.5–8.7**

Table 2: Expected performance ranges on CoReQA (Qwen-7B as generator). "Reported" values from Chen et al. [2025].

7 Discussion

7.1 How Data and Approach Address the RQs

RQ1 is evaluated by comparing the full RepoQA system against the BM25-only baseline on all 1,563 CoReQA pairs. The four-dimensional LLM-as-a-judge evaluation provides nuanced insight: we expect hybrid retrieval to most improve *completeness* (by retrieving context from multiple related files) and *accuracy* (by providing the correct code definitions rather than superficially similar text).

RQ2 is evaluated by comparing question-type-routed retrieval against uniform retrieval, with CoReQA's questions manually categorized into architecture/logic/deployment types. The expectation is the largest gains on architecture questions, where the hierarchical strategy (project $\rightarrow$ directory $\rightarrow$ file) provides crucial structural context that a flat retrieval misses.

RQ3 is evaluated through the ablation study (Table 1), quantifying each component's marginal contribution. The hypothesis that hierarchical summaries contribute most to completeness, hybrid fusion contributes most to accuracy, and query expansion contributes most to relevance.

7.2 Broader Implications

If the hybrid retrieval approach demonstrates significant improvement over BM25, this has immediate practical implications for AI coding assistants. The same pipeline could be integrated into IDE chat interfaces (Cursor, JetBrains AI), documentation generators, and developer onboarding tools.

The hierarchical summarization layer is also independently valuable: it produces human-readable project documentation as a byproduct of the indexing process.

8 Summary of Progress

8.1 Completed

- Literature review of 8 core papers spanning code QA, RAG for code, and repository-level understanding.
- System architecture design with four-layer pipeline.
- Technology stack selection: Python, Tree-sitter, ChromaDB, LangChain, OpenAI/Claude APIs.
- CoReQA benchmark downloaded and analyzed (20+ QA pairs examined in detail).
- Question taxonomy defined (architecture / logic / deployment) with examples from CoReQA.
- Evaluation framework defined using CoReQA's LLM-as-a-judge with ablation study design.
- Project repository initialized with README and environment configuration.

8.2 Remaining Work

- Repository ingestion pipeline (Tree-sitter parsing, AST-aware chunking).
- Hierarchical summarization prompts and generation pipeline.
- Embedding pipeline and ChromaDB integration.
- Hybrid retrieval engine with RRF fusion.
- Question classifier and type-aware routing.
- Structured prompt templates and hallucination checker.
- Web UI (Streamlit) with source reference linking.
- Full CoReQA benchmark evaluation.
- Ablation study and user study (3–5 developers).
- Final report and presentation.

8.3 Risks and Mitigation

- **LLM API costs:** Mitigated by caching embeddings and summaries; using cheaper models during development.
- **Hierarchical summarization quality:** Iterative prompt refinement with manual validation on 3–5 repos.
- **Evaluation variance:** Running LLM-as-a-judge 3–5 times per evaluation and reporting mean $\pm$ std.
- **CoReQA data access:** Dataset is publicly available; repo cloning may face rate limits (mitigated by pre-cloning).

References

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Ilya Sutskever. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

Rishab Bairi, Abhishek Sonwane, Aditya Kanade, Arun Iyer, Sriram Parthasarathy, Sriram Rajamani, B. Ashok, and S. Shet. CodePlan: Repository-level coding using LLMs and planning. In *Proceedings of the ACM on Software Engineering (FSE)*, pages 675–698, 2024.

CAST Authors. CAST: Enhancing code retrieval-augmented generation with structural chunking via abstract syntax tree. In *Findings of EMNLP 2025*. Association for Computational Linguistics, 2025.

J. Chen, K. Zhao, J. Liu, C. Peng, J. Liu, H. Zhu, P. Gao, P. Yang, and S. Deng. CoReQA: Uncovering potentials of language models in code repository question answering. *arXiv preprint arXiv:2501.03447*, 2025.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harrison Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

W. Cheng, Y. Wu, and W. Hu. Dataflow-guided retrieval augmentation for repository-level code completion. In *Proceedings of ACL 2024*, pages 7957–7977, 2024.

CodeRepoQA Authors. CodeRepoQA: A large-scale benchmark for software engineering question answering. *arXiv preprint arXiv:2412.14764*, 2024.

Yunfan Gao, Yun Xiong, Xiaozhi Gao, Kangxiang Jia, Jinjin Pan, Yuxuan Bi, Yi Dai, Jiawei Sun, Haofen Wang, and Hong Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2024.

Gistify Authors. Gistify! codebase-level understanding. *OpenReview preprint*, 2025.

Carlos E. Jimenez, John Yang, Alexander Wettig,

Shunyu Yao, K. Pei, Omer Press, and Karthik R. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *Proceedings of ICLR 2024*, 2024.

C. Lee, Y. Seonwoo, and A. Oh. CS1QA: A dataset for assisting code-based question answering in introductory programming course. *arXiv preprint arXiv:2210.14494*, 2022.

C. Liu and X. Wan. CodeQA: A question answering dataset for source code comprehension. In *Findings of EMNLP 2021*, pages 2618–2632, 2021.

Y. Peng et al. SWE-QA: Can language models answer repository-level code questions? *arXiv preprint arXiv:2509.14635*, 2025.

J. Strich, F. Schneider, I. Nikishina, and C. Biemann. On improving repository-level code QA for large language models. In *Proceedings of ACL 2024 Student Research Workshop*, pages 209–244, 2024.

A. Tufek et al. Repository-level code understanding by LLMs via hierarchical summarization. In *ICCSA 2025*. Springer, 2025.

Z. Z. Wang, A. Asai, X. V. Yu, F. F. Xu, Y. Xie, G. Neubig, and D. Fried. CodeRAG-Bench: Can retrieval augment code generation? In *Findings of NAACL 2025*, pages 3199–3214, 2025.

Xin Xia, Lingfeng Bao, David Lo, Zhenchang Xing, Ahmed E. Hassan, and Shanping Li. Measuring program comprehension: A large-scale field study with professionals. *IEEE TSE*, 44 (10):951–976, 2018.

Fengji Zhang, Bei Chen, Yan Zhang, Jacky Keung, J. Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. RepoCoder: Repository-level code completion through iterative retrieval and generation. In *Proceedings of EMNLP 2023*, pages 2471–2484, 2023.

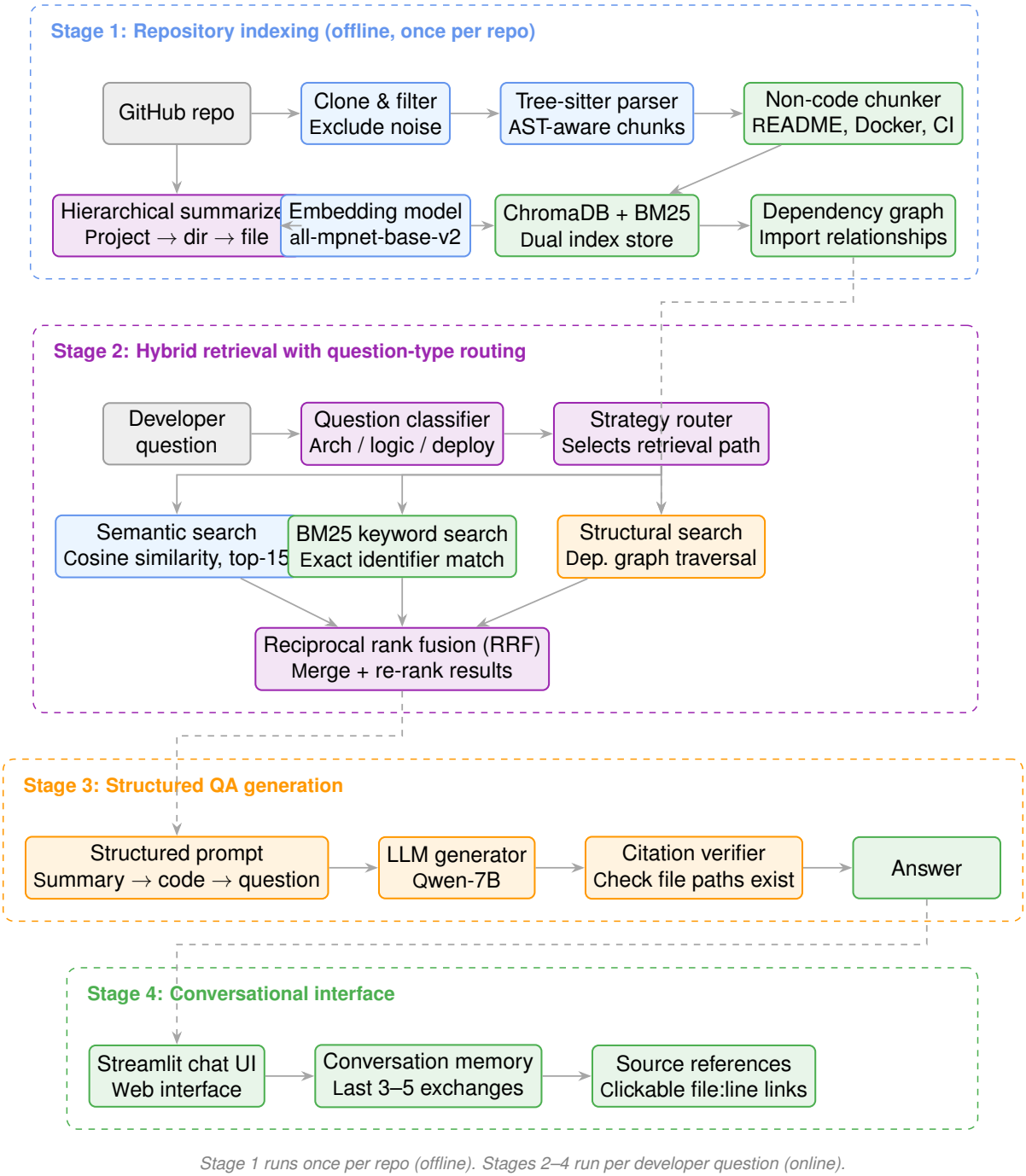

Figure 1: Architecture of the RepoQA system. Stage 1 performs offline indexing with hierarchical summarization. Stage 2 retrieves relevant context using hybrid search with question-type routing. Stage 3 generates grounded explanations with citation verification.